Bicol University, Polangui
Computer Studies Department

Final Project Proposal

Group Name: Hexa Odyssey

Course: IS Elec 2 – Web Systems and Technologies

Section: BSIS - 2A

Members:

Ailes, Estrylle Jazmyne B. **Boro, Elyza Marie D.** **Dealino, Maulana Mahdi S.**

Nicolas, Pauline Kaye B. **Orellana, Neil Angelo N.** **Ruiz, Jasmine R.**

Project Title: DAGOS
A web app for real-time fare tracking and map routing in Bicol.

Introduction
DAGOS is a web app that shows commuters and tourists in Bicol the fair cost of a trip from one point to another, in real time. It pairs a map of Bicol's places and transport stops with the fare for each route, so riders know what they should pay before they get on a bus, jeep, van, or tricycle.

System Purpose
DAGOS exists to give every rider in Bicol a real-time, true fare for the trip they are about to take. Its map routing lets tourists plan a route and check the fare for it, so they are not scammed by an inflated price.

Target Users
DAGOS is built for people who ride public transport in Bicol.

- **Commuters:** regular riders of buses, jeepneys and similar transport who want to know the right fare for their route.

- **Tourists:** visitors who don't know local fares and are most at risk of being overcharged.

Key Features

1. **Bicol transport map:** a map of places in Bicol that marks the bus stops, jeepney stops and other transport points.
2. **Point A to B fares:** pick a starting point and a destination and see the fare cost for that trip.
3. **Real-time fares:** fare information is kept current, so riders see the true cost and not an outdated one.
4. **Map routing:** the route from point A to point B is drawn on the map.
5. **Tourist fare protection:** tourists can check the fair fare before paying, so they can spot and avoid overcharging.

Use Case Diagram (UML)

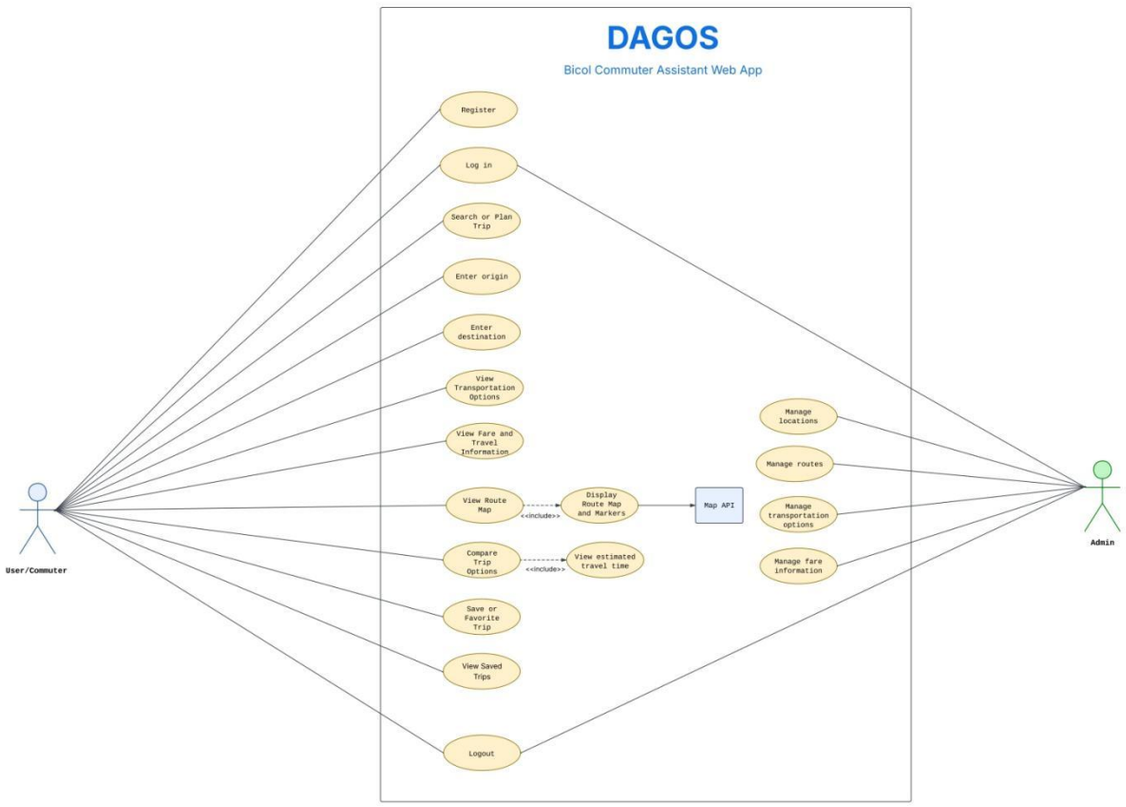


Figure 1. Use Case Diagram of DAGOS: Bicol Commuter Assistant Web App

The use case diagram shows how the two main actors, the User/Commuter and the Admin, interact with the DAGOS system. It also shows the external Map API the system relies on.

The **User/Commuter** is the main user of the system. A commuter can register and log in to access the app's features. Once logged in, the user can search or plan a trip by entering an origin and a destination. The system then lets the user view the available transportation options as well as the fare and travel information for the trip. The user can view the route on a map, which the system displays with markers through the Map API, and compare trip options, which includes viewing the estimated travel time of each. Commuters can also save or favorite trips and view them later, then log out when finished.

The **Admin** is responsible for maintaining the data the system depends on. After logging in, the admin can manage locations, routes, transportation options, and fare information, which keeps the information shown to commuters accurate and up to date. The admin logs out after managing the system's records.

The diagram also uses two *include* relationships. "View Route Map" includes "Display Route Map and Markers," which depends on the **Map API** to show locations and routes. "Compare Trip Options" includes "View Estimated Travel Time," so every comparison shows the estimated time for each option.

Overall, the diagram shows that DAGOS separates the commuter's trip-planning functions from the admin's data-management functions

Entity-Relationship Diagram

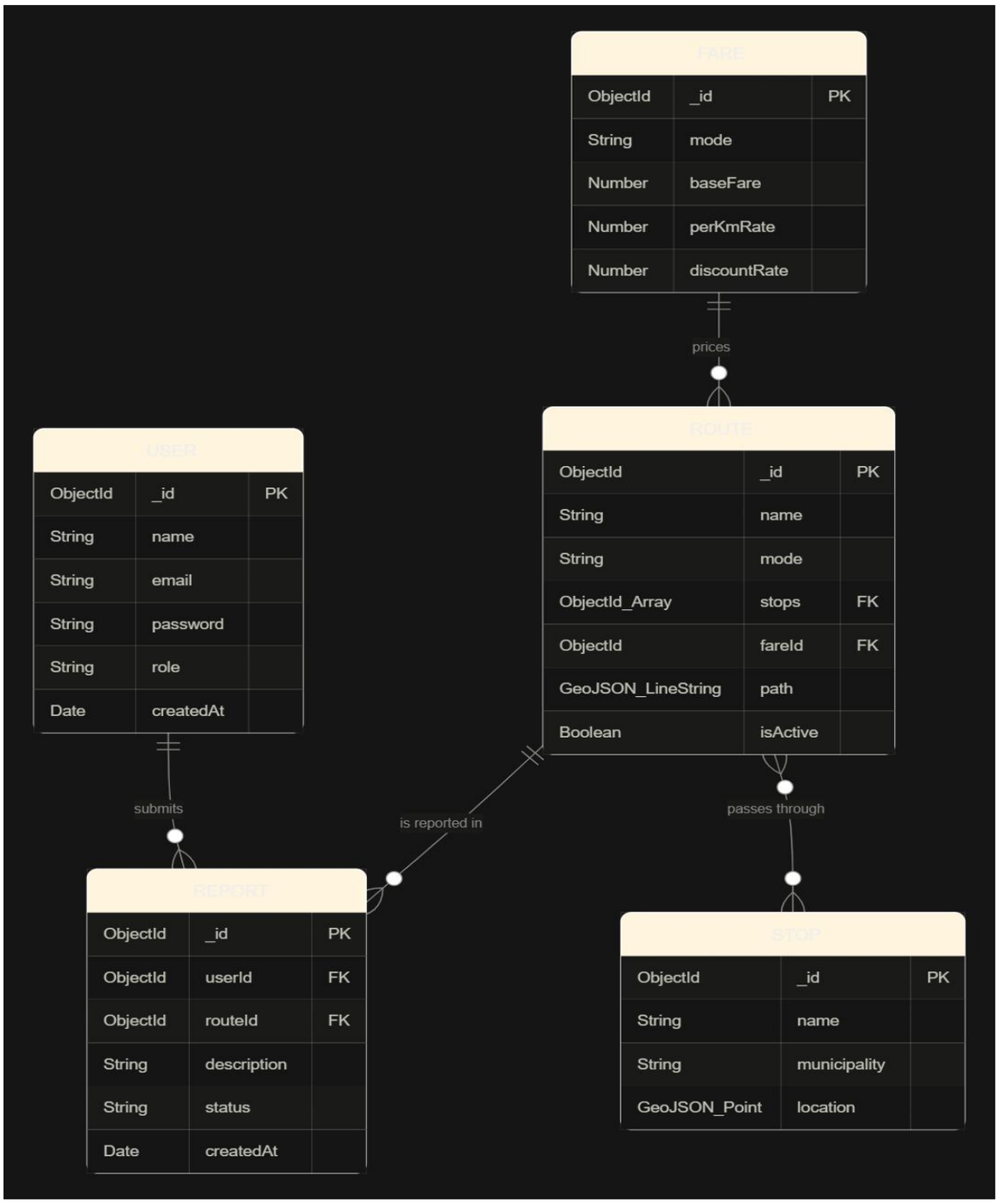


Figure 3. Entity Relationship Diagram (ERD) of DAGOS

Main API Endpoints

- POST /api/auth/register and POST /api/auth/login
- GET /api/stops/nearby?lat=&lng=&radius=
- POST /api/routes/search (origin and destination in, route options and fare out)
- GET /api/routes and GET /api/routes/:id
- POST /api/routes, PUT /api/routes/:id, DELETE /api/routes/:id (admin only)
- POST /api/reports (logged-in users)
- PUT /api/reports/:id (admin approves or rejects)

Data Sourcing
Route and fare data will be collected from LTFRB Region V route and fare listings, OpenStreetMap, and our own field surveys (GPS tracing and rider interviews along main corridors). Data will start with a limited set of routes in the Legazpi-Daraga area and expand to other towns as time allows.

Security and Data Handling
Passwords will be hashed before storage, protected routes will require a valid token, and all user input will be validated on the server. Environment variables will store sensitive configuration and will not be committed to GitHub.

Backend Responsibilities

- Design the MongoDB collections and the ERD
- Develop the Express routes, controllers, and route-search logic
- Import and maintain the route, stop, and fare data
- Connect the backend to the database and the frontend
- Test the endpoints and fix backend bugs
- Maintain the /backend and /database folders in the repository

Backend Development Plan

Backend Overview
The backend of Dagos will be built with Node.js and Express, with MongoDB as the database. It stores the stops, routes, and fares of public transport in Albay (focusing on Legazpi, Daraga, Tabaco, and Ligao). It handles route searching, fare estimation, user accounts, and commuter-submitted route corrections, and it provides REST API endpoints to the frontend.

Technology Stack

6. Runtime and framework: Node.js, Express.js
7. Database: MongoDB with Mongoose, using 2dsphere geospatial indexes
8. Authentication: JWT, with bcrypt for password hashing
9. Other tools: Thunder Client for API testing, dotenv for configuration, OpenStreetMap/Nominatim for place search and map data

Backend Features

1. **User authentication and authorization:** registration, login, and role-based access (Commuter, Contributor, Admin).
2. **Stop and route management:** admins can create, read, update, and delete jeepney, UV Express, bus, and tricycle stops and routes.
3. **Route search:** given an origin and destination, find nearby stops (within a set walking distance) and return the best route options, including transfers.
4. **Fare estimation:** compute the fare per route and mode, with discounts for students, seniors, and PWDs.
5. **Commuter reports and admin verification:** users can report wrong, missing, or changed routes, and admins approve or reject them before the data is updated.

Database Design (Collections)

Collection	Key Fields	Relationship
User	_id (ObjectId), name (String), email (String), password (String), role (String), createdAt (Date)	One-to-Many with Report
Stop	_id (ObjectId), name (String), municipality (String), location (GeoJSON Point)	Many-to-Many with Route
Route	_id (ObjectId), name (String), mode (String), stops (Array of ObjectId), path (GeoJSON LineString), isActive (Boolean)	References Stop and Fare
Fare	_id (ObjectId), mode (String), baseFare (Number), perKmRate (Number), discountRate (Number)	One-to-Many with Route
Report	_id (ObjectId), userId (ObjectId), routeId (ObjectId), description (String), status (String), createdAt (Date)	Belongs to User and Route

Data Flow Diagram

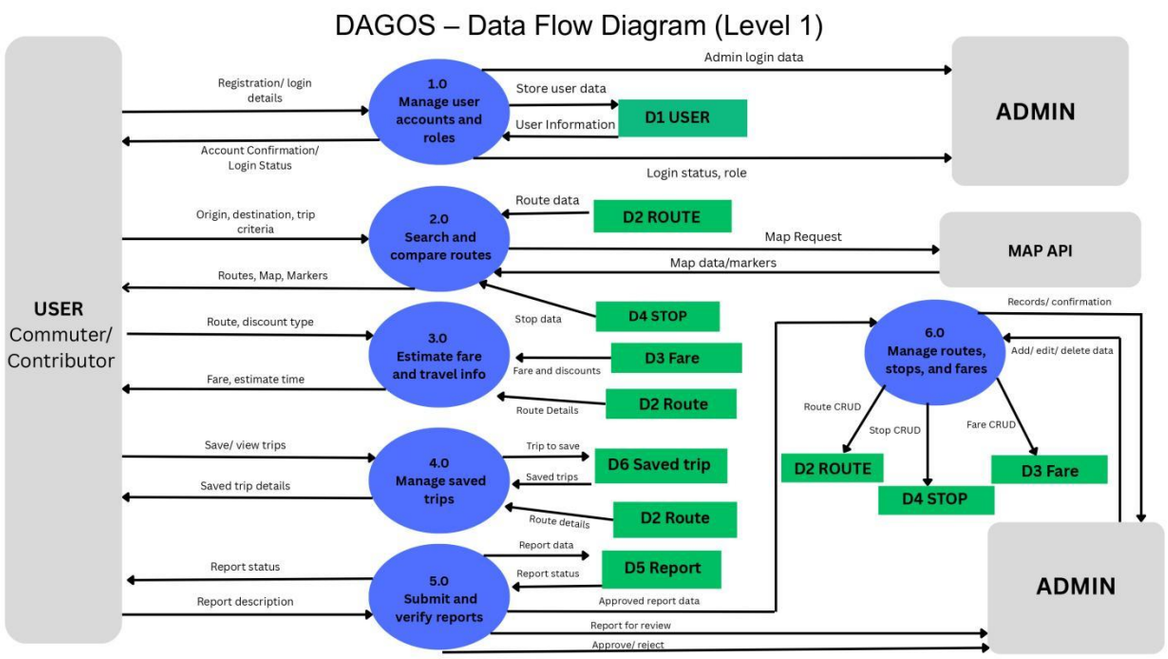


Figure 3. Data Flow Diagram (Level 1) of DAGOS

The Level 1 Data Flow Diagram breaks DAGOS into six main processes. It shows how data moves between these processes, the three external entities (the User/Commuter/Contributor, the Admin, and the Map API), and the six data stores (D1 User, D2 Route, D3 Fare, D4 Stop, D5 Report, and D6 Saved Trip).

Process 1.0, Manage User Accounts and Roles, handles registration and login. The user submits registration or login details, and the process stores them in D1 User and retrieves user information to verify them. It then returns an account confirmation or login status. The process also passes login status and role information to the Admin, so the system can tell commuters and administrators apart.

Process 2.0, Search and Compare Routes, receives the user's origin, destination, and trip criteria. It gathers route data from D2 Route and stop data from D4 Stop, and sends a map request to the Map API, which returns map data and markers. The user then receives the matching routes with the map and markers.

Process 3.0, Estimate Fare and Travel Info, takes the selected route and the user's discount type. It retrieves fare and discount information from D3 Fare and route details from D2 Route, then returns the estimated fare and travel time to the user.

Process 4.0, Manage Saved Trips, lets users save and view trips. A trip to save is stored in D6 Saved Trip. When the user views their trips, the process retrieves the saved trips along with their route details from D2 Route and displays the saved trip details.

Process 5.0, Submit and Verify Reports, lets contributors report problems or updates, such as wrong routes or outdated fares. The report description is stored in D5 Report, and the report is sent to the Admin for review. The Admin approves or rejects it, and the user is notified of the report status. Approved report data is passed on to Process 6.0.

Process 6.0, Manage Routes, Stops, and Fares, is the Admin's maintenance process. The Admin adds, edits, or deletes data, and the process applies these changes (Route, Stop, and Fare CRUD) to D2 Route, D4 Stop, and D3 Fare. It also uses the approved report data from Process 5.0 and returns a confirmation to the Admin.

Overall, the diagram shows that DAGOS keeps its information accurate through two channels: administrator management and verified reports from users.

Github Repository URL:

https://github.com/lyshie12/final-project-HexaOdyssey-BSIS2a

